



ИИ в программировании: от «нажать и сработает» до сознательного инженера

«Короче, чуваки. Лекция — как инструкция к дрели: можно нажать и получить дырку, а можно понять, куда сверлить, чтобы не тронуть проводку.»

Коротко: поймём, какие ИИ-инструменты есть, как их использовать без фейлов и вайб-кодинга, и почему ваш мозг всё ещё важнее любого агента. Теплый, дерзкий тон, реальные советы — для первокурсников и джуниоров.

Введение: зачем вам вообще этот ИИ-шум?

ИИ уже не только про генерацию текста — это экосистема: изображение, звук, анализ медицинских снимков, прогнозы спроса, автоматизация процессов. Для вас это значит: умение интегрировать модели, кастомизировать их и — самое важное — не стать их заложником.



Генерация медиа

Картинки, видео и дизайн из текста — теперь часть процесса разработки UI/UX.

Код и помощники

От автодополнения в IDE до агентов, которые могут чинить баги — но не всегда правильно.

Специализированные системы

ИИ решает узкие задачи (диагностика, логистика) — и это ваша будущее поле для интеграции.

Классификация инструментов: не теряйтесь в названиях

Ниже — три практичных типа, которые вы встретите в универе и на стажировке. Я даю рекомендации, что выбрать первокурснику и почему.

Autocompletion (Copilot, Codeium)

Встраивается в IDE, подсказывает строки/функции — удобно для рутинных задач. Совет: всегда читаем и переписываем предложенное.

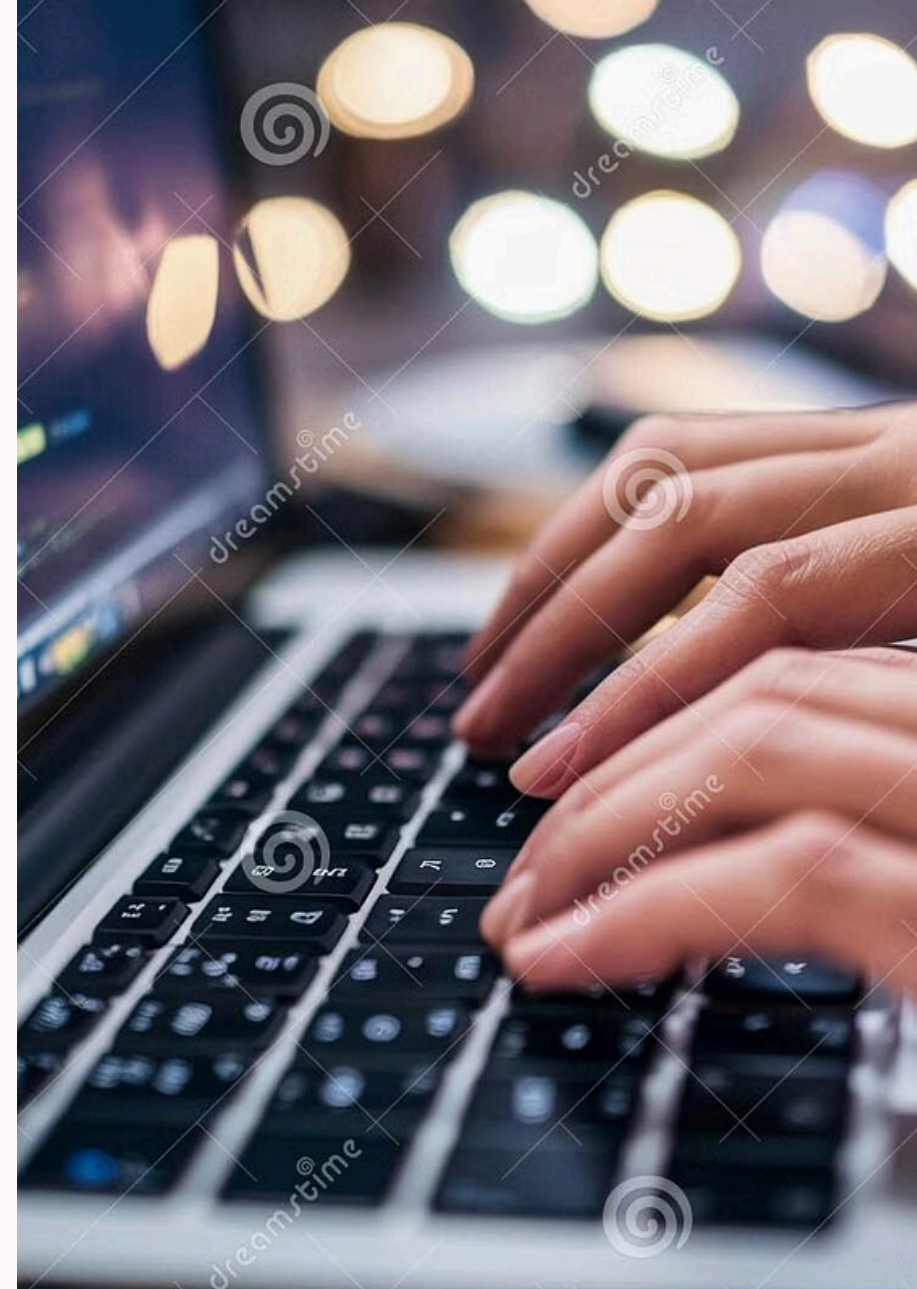
Чат-модели (GPT, Claude)

Удобны для объяснений, написания тестов и рефакторинга. Используйте для разбора концепций, а не для слепого копипаста.

Локальные модели (Ollama, Llama.cpp)

Для приватности и экспериментов без интернета — идеальны, если не хотите сливать NDA код в облако.

Рекомендация: для повседневного — Codeium; для глубокой аналитики — Claude/ChatGPT; для приватных проектов — локальная модель.



Промпт-инжиниринг: RODE – ваша новая привычка

Хороший промпт – это 80% результата. Формула RODE: Role, Objective, Details, Examples. Без неё получаете «мусорный» код, который выглядит красиво, но падает на граничных случаях.

01

Role – кто модель?

«Ты – строгий преподаватель, объясняющий каждой строчке» – меняет тон и глубину объяснения.

02

Objective – что нужно

Четкая цель: «Напиши функцию сортировки слиянием с пояснениями и тестами».

03

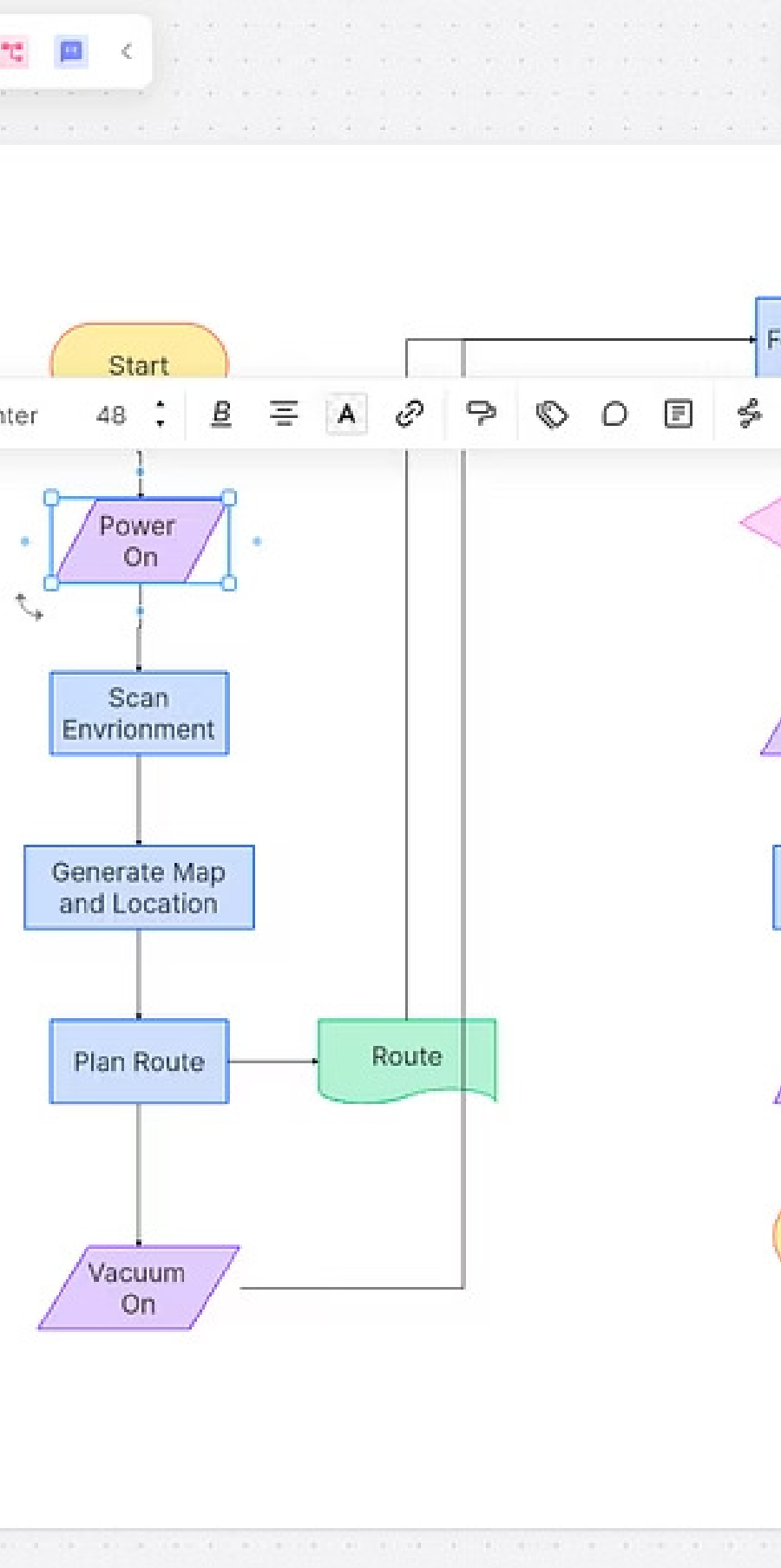
Details – условия

Ограничения: стандартная библиотека, отсутствие рекурсии, формат вывода JSON и т.д.

04

Examples – few-shot

Показывайте 2–3 примера желаемого ответа – модель быстрее схватит паттерн.



Chain-of-Thought и few-shot – заставляем модель думать

Добавляйте инструкцию «сначала опиши план шаг за шагом, затем напиши код». Few-shot примеры ускоряют корректные ответы. Для алгоритмов – просите сначала объяснить, потом кодировать.



Эта последовательность резко сокращает логические ошибки и повышает шанс, что код пройдет базовые тесты.

The Project Planning Process in 5 Steps





PA
NO
PENSAR EN TI
TENGO
QUE
FUMAR

bad bunny - ni bien ni mal

Агенты: когда ИИ делает многошаговые задачи

Агент — это цикл мысль→действие→наблюдение. Пример: добавить endpoint в Flask — агент просматривает проект, добавляет модель, обновляет requirements, запускает тесты и фиксирует ошибки. Круто, но опасно: агенты могут расходовать API-бюджет и заикливаться.



Инструменты

Файловая система, терминал, pip, git — агент ими оперирует.



Фреймворки

LangChain, AutoGPT, CrewAI — стартовый набор для сборки агентов.



Ограничения

Заикливание, неконтролируемые коммиты, безопасность — нужен человеческий надзор.



Тёмная сторона: галлюцинации и вайб-кодинг

Генеративные модели не хранят истину — они предсказывают слова. Из этого вытекают: вымышленные библиотеки, неверные параметры, неучтённые граничные случаи. Вайб-кодинг — когда вы доверяете модели на 100% и не понимаете, что сделали.

Галлюцинации

Модель может придумать несуществующий модуль или документацию — проверяйте официальные источники.

Безопасность

Риск внедрить уязвимости: SQL-инъекции, жёстко закодированные ключи, устаревшие паттерны.

Этика

Нельзя отправлять в облако пароли, секреты или код под NDA — используйте локальные модели.

Правила безопасности мышления – три простых шага

Не делайте коммит, если не можете объяснить каждую строку. Правила: проверяй всегда, сверяйся с документацией, используй ИИ как ускоритель, а не замену мышления.

1. Понимай код: переименовывай переменные, переписывай логические блоки своими словами.
2. Проверь граничные случаи и автоматизируй тесты (unit + edge cases).
3. Верифицируй методы в официальной документации перед использованием.



Карьера в мире, где код пишут машины – три уровня

Где вы будете завтра? Подумайте по уровням:



Новичок-потребитель

Просишь ИИ «сделай», но не понимаешь. Риск: провал на экзаменах и интервью.



Осознанный разработчик

Проектируешь архитектуру, контролируешь качество, используешь ИИ для рутинных задач.



Создатель агентов

Собираешь сети агентов, управляешь их работой и отвечаешь за результаты. Высший пилотаж.

Цель на 4-й курс — стабильно второй уровень, а дальше — брать на себя третий.



Итог и задание

ИИ — мощный инструмент, но без фундамента он приведёт к аварии.
Правило-мантра: «Понимай, проверяй, автоматизируй». Немного дерзко: не будьте тем чуваком, который коммитит и улыбается — будьте тем, кто знает, почему он это сделал.

Практическое домашнее задание (для тех, кто не боится):

1. Возьмите кусок кода (100–200 строк). Спросите у модели: «Объясни каждую функцию в 2–3 предложениях». Затем перепишите одну функцию своими словами и напишите 3 unit-теста.
2. Составьте RODE-промпт и попросите модель сгенерировать реализацию. Проверьте её на краевых случаях и опишите, где модель могла галлюцинировать.
3. Подготовьте короткое сообщение в чат для обсуждения: «Что ИИ сделал хорошо и где он подвёл?» — принесите на следующее занятие.